Université Paris-Saclay
Master 1 MIAGE · UE DSID
Année universitaire 2026-2027

Projet Big Data

Rapport : des sources brutes à la couche Silver (API, CSV et CRM avec Airbyte, MinIO et PostgreSQL)

ADEL Melina

M1 MIAGE en apprentissage

Apprentie CISO Office PMO
chez CEVA Logistics (groupe CMA CGM)

Enseignant : M. Antunes
Rendu le 13 novembre 2026
Dossier technique associé : `ADEL_MELINA_PROJET_BIG_DATA`

Remerciements

Je remercie M. Antunes pour son cours et pour l'idée de ce projet. Il m'a permis d'apprendre à construire une chaîne de données du début à la fin, de la collecte des données brutes jusqu'à une couche Silver testée, et de manipuler moi-même des outils que je ne connaissais pas, comme Airbyte et MinIO.

Je réalise ce master en apprentissage : je suis apprentie PMO au sein du CISO Office de CEVA Logistics (groupe CMA CGM). J'ai donc mené ce projet en parallèle de mon travail en entreprise.

Sommaire

Présentation rapide du résultat

1. Architecture
2. Sources
3. Bronze
4. Silver
5. Tests
6. CRM delta
7. Docker / Airbyte
8. Reproductibilité

Présentation rapide du résultat

Le projet demande de construire une chaîne de données qu'on peut rejouer, à partir de trois sources de natures différentes : une API publique, un fichier CSV et le CRM EspoCRM. Les données doivent d'abord arriver telles quelles dans une couche Bronze de PostgreSQL, puis être nettoyées et typées dans une couche Silver écrite en SQL, avec des tests.

J'ai tout fait tourner dans un GitHub Codespace, parce que mon ordinateur n'avait pas assez de mémoire (je l'explique dans la partie 8). Le tableau ci-dessous résume où j'en suis au moment du rendu : les trois flux fonctionnent de bout en bout et les 7 vues Silver passent les 8 tests obligatoires.

Tableau 1 – Résultat final des trois flux

Flux	Source	Table(s) Bronze	Vue(s) Silver	Lignes Bronze → Silver
A	API Frankfurter (taux de change de la BCE)	`bronze.api_frankfurter`	`silver.api_frankfurter_rates`	21 → 21
B	CSV des communes de France (data.gouv.fr)	`bronze.csv_communes`	`silver.csv_communes`	34 868 → 34 868
C	EspoCRM / MariaDB	`bronze.account`, `contact`, `opportunity`, `account_contact`, `contact_opportunity`	`silver.crm_account`, `crm_contact`, `crm_opportunity`, `crm_account_contact`, `crm_contact_opportunity`	1, 2, 1, 2 et 2 lignes, identiques des deux côtés

Le dossier technique contient le Compose, les scripts SQL, les sorties des tests, les manifestes et les captures. Il ne contient aucun mot de passe : seul le fichier `compose/.env.example` est versionné, le vrai `.env` est ignoré par Git.

1. Architecture

1.1 Schéma global

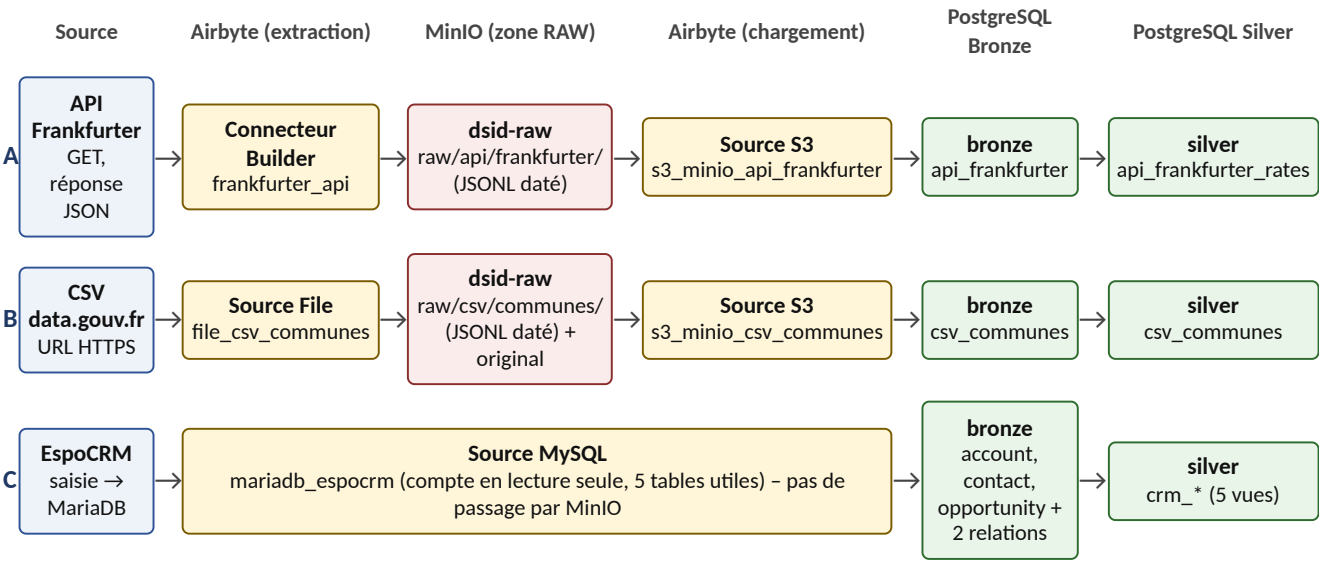


Figure 1 – Les trois chemins d'ingestion et la même convention Bronze → Silver à l'arrivée

Les flux A et B suivent le même chemin. Une première connexion Airbyte lit la source et dépose la donnée brute dans MinIO, dans la zone RAW. Une deuxième connexion relit ces fichiers avec la source S3 et les charge dans le schéma `bronze` de PostgreSQL. Le flux C est plus direct : Airbyte lit la base MariaDB d'EspoCRM avec la source MySQL et charge tout de suite dans `bronze`. Dans les trois cas, la couche Silver est un ensemble de vues SQL dans le schéma `silver`.

Le passage par MinIO peut ressembler à un détour, mais il sert à garder une copie datée de ce qui a été reçu. L'API renvoie de nouveaux taux tous les jours et le fichier de data.gouv.fr peut être mis à jour : si je recharge Bronze plus tard, le lot d'origine reste disponible dans MinIO avec son empreinte SHA-256.

1.2 Rôle de chaque composant

Tableau 2 – Composants et rôles

Composant	Version	Rôle dans le projet
Docker Compose	fichier `compose/compose.yaml`	Lance PostgreSQL, MariaDB, EspoCRM, MinIO et Adminer avec des versions figées.
PostgreSQL	18.6	Entrepôt : schéma `bronze` (données reçues) et schéma `silver` (vues standardisées).
MariaDB	11.8.9	Base de données d'EspoCRM. Airbyte la lit avec un compte en lecture seule.
EspoCRM	10.0.8	CRM dans lequel je saisis les nouvelles données (compte, contacts, opportunité).
MinIO	RELEASE.2025-10-15T17-29-55Z	Stockage objet compatible S3 : conserve les lots bruts et le CSV original.
Airbyte	2.5.0, installé avec abctl v0.30.4	Outil d'ingestion : 5 connexions (partie 3). Il tourne dans un cluster kind, à côté du Compose.
Adminer	6.1.0	Console web pour consulter PostgreSQL et MariaDB.
GitHub Codespaces	Linux, 4 cœurs, 16 Go	Machine sur laquelle tout est exécuté.

2. Sources

Avant de configurer Airbyte, j'ai décrit pour chaque source ce que représente une ligne (le grain), quelle est la clé, quelles sont les limites et ce que j'ai le droit de faire avec les données.

2.1 API publique : Frankfurter (taux de change, source BCE)

L'API Frankfurter fournit des taux de change quotidiens. Je l'ai choisie parce qu'elle est publique, documentée, qu'elle ne demande ni clé ni compte et qu'elle ne contient aucune donnée personnelle.

Tableau 3 – Contrat de l'API

Question	Réponse
Objet métier	Taux de change de référence de l'euro vers USD, GBP et JPY
Appel GET	`https://api.frankfurter.dev/v2/rates?from=2026-11-02&to=2026-11-10&base=EUR&quotes=USD,GBP,JPY&providers=ECB`
Format de réponse	Tableau JSON d'objets plats : `date`, `base`, `quote`, `rate`
Grain	Un taux pour une date et une devise cotée
Clé	(`date`, `base`, `quote`)
Types attendus	`date` : date ; `base` et `quote` : texte ; `rate` : décimal
Pagination	Non documentée : une requête renvoie toute la plage demandée. Je contrôle la complétude avec le nombre de lignes (jours ouvrés × 3 devises).
Fraîcheur	Quotidienne. Il n'y a pas de ligne pour les week-ends (les 7 et 8 novembre sont absents).
Limites de débit	Aucune limite documentée. Un code 503 peut signaler une requête trop lourde ou un serveur saturé (en-tête `Retry-After`).
Données personnelles, secret	Aucune donnée personnelle, aucun secret
Droits d'utilisation	La documentation présente un service gratuit, open source et sans authentification, mais ne précise pas de licence pour les données. Les taux viennent de la Banque centrale européenne.

Pourquoi le paramètre `providers=ECB`. Sans lui, la version 2 de l'API renvoie des taux combinés entre plusieurs fournisseurs. Avec lui, les valeurs sont celles de la BCE : j'ai vérifié qu'elles sont égales à celles de la version 1 pour le 10 novembre 2026 (GBP 0,83545 ; JPY 177,38 ; USD 1,1214). La source est donc figée et on peut la rejouer.

Résultat du GET du 13 novembre 2026 : 21 lignes, soit 7 jours ouvrés × 3 devises, du 2 au 10 novembre 2026. Un échantillon de la réponse est conservé dans `data/api_frankfurter_sample.json`, avec son SHA-256 dans `data/manifestes/`.

2.2 Fichier CSV : liste des communes de France

Le fichier est la liste des communes de France publiée sur data.gouv.fr, avec 62 colonnes par commune (codes, noms, population, superficie, coordonnées…).

Tableau 4 – Contrat du fichier CSV

Question	Réponse
Origine	data.gouv.fr, jeu « Liste des communes de France : code INSEE, codes postaux, EPCI, population, superficie (62 indicateurs) », mis à jour le 17 juin 2026
URL lue par Airbyte	`https://www.data.gouv.fr/api/1/datasets/r/c63fd0b1-7987-46f6-b779-8b3ed889090c` (URL stable de la ressource)
Licence	Licence Ouverte / Open Licence 2.0 : réutilisation libre en citant la source

Format	CSV, séparateur virgule, en-tête sur la première ligne, champs entre guillemets quand ils contiennent une virgule
Encodage	UTF-8 (vérifié avec la commande `file`)
Taille	21 344 470 octets ; 34 869 lignes en comptant l'en-tête, donc 34 868 communes
Colonnes	62 (`code_insee`, `nom_standard`, `reg_code`, `dep_code`, `population`, `superficie_km2`, `latitude_mairie`…)
Grain	Une commune par ligne
Clé	`code_insee`, 5 caractères. Je le garde en texte : `01001` commence par un zéro et la Corse utilise `2A` et `2B`.
Valeurs vides	Il y en a (par exemple `code_insee_centre_aire_attraction` est vide pour la commune 01002).
SHA-256 du fichier	`f15a1dffa3105d267a4aaa4e3e7b161fea485187a6726bdf59247a94c70552e6`
Données personnelles	Aucune : données géographiques et statistiques sur des communes

2.3 CRM : EspoCRM et sa base MariaDB

EspoCRM enregistre ses données dans la base MariaDB `espocrm`. La découverte Airbyte y a trouvé 141 tables. Je n'en ai gardé que 5, celles qui servent à mon cas : un compte, ses contacts et une opportunité liée. Les noms physiques viennent de la découverte Airbyte et d'une requête sur `information_schema` dans MariaDB.

Tableau 5 – Tables utiles du CRM

Table MariaDB	Contenu et grain	Clé	Curseur possible
`account`	Une ligne par compte (société)	`id`, texte de 17 caractères	`modified_at`
`contact`	Une ligne par contact (personne)	`id`	`modified_at`
`opportunity`	Une ligne par opportunité	`id`	`modified_at`
`account_contact`	Une ligne par lien compte ↔ contact	`id` (entier auto-incrémenté) ; couple (`account_id`, `contact_id`)	Aucun : pas de colonne de date
`contact_opportunity`	Une ligne par lien contact ↔ opportunité	`id` ; couple (`contact_id`, `opportunity_id`)	Aucun

Les deux tables de relation sont nécessaires à mon cas. Le compte d'un contact ou d'une opportunité se lit déjà dans la colonne `account_id`, mais le lien entre un contact et une opportunité n'existe que dans `contact_opportunity`. La table `account_contact` porte le rôle du contact dans le compte.

Deux points à connaître sur cette source. EspoCRM ne supprime pas physiquement une fiche, il passe sa colonne `deleted` à 1. Et les dates `created_at` et `modified_at` sont enregistrées en UTC, sans fuseau.

Compte Airbyte. Airbyte se connecte avec l'utilisateur MariaDB `airbyte`, créé au premier démarrage par le script `compose/init/mariadb/01_airbyte_user.sh`, qui n'a que le droit `SELECT` sur la base `espocrm`. Son mot de passe est dans `compose/.env`, qui n'est pas versionné.

Droits d'utilisation. Ce sont des données fictives que j'ai créées pour le test, sans aucune donnée personnelle réelle.

3. Bronze

3.1 Configuration Airbyte

J'ai créé 5 connexions, toutes en déclenchement manuel. Les noms du tableau sont ceux qu'on retrouve dans l'interface d'Airbyte. J'ai relu les modes de synchronisation directement dans la base de configuration d'Airbyte pour être sûre de décrire ce qui est réellement configuré.

Tableau 6 – Les 5 connexions Airbyte

Connexion	Source → destination	Flux (stream)	Mode de synchronisation	Volume	Statut
`api_frankfurter_to_minio`	`frankfurter_api` (connecteur créé dans le Builder) → `minio_raw` (S3)	`frankfurter_api`	Full refresh | Append	21	Réussi
`s3_minio_to_postgres_api`	`s3_minio_api_frankfurter` (S3 v4.17.1) → `postgres_bronze`	`api_frankfurter`	Incremental | Append, curseur `_ab_source_file_last_modified`	21	Réussi
`csv_communes_to_minio`	`file_csv_communes` (File) → `minio_raw_csv` (S3)	`communes_france`	Full refresh | Append	34 868	Réussi
`s3_minio_to_postgres_csv`	`s3_minio_csv_communes` (S3 v4.17.1) → `postgres_bronze`	`csv_communes`	Incremental | Append, curseur `_ab_source_file_last_modified`	34 868	Réussi
`mariadb_espocrm_to_postgres_crm`	`mariadb_espocrm` (MySQL) → `postgres_bronze`	5 tables (partie 2.3)	Incrémental pour les 3 entités, Full refresh pour les 2 relations (détail partie 6)	1, 2, 1, 2, 2	Réussi

Tableau 7 – Paramètres des sources et destinations (sans les secrets)

Nom	Type	Paramètres principaux
`frankfurter_api`	Source, Connector Builder	GET sur l'URL de la partie 2.1 ; clé primaire (`date`, `quote`)
`minio_raw`	Destination S3	Bucket `dsid-raw`, chemin `raw/api/frankfurter`, endpoint `http://172.19.0.1:9000`, format JSONL
`s3_minio_api_frankfurter`	Source S3	Bucket `dsid-raw`, glob `raw/api/frankfurter/frankfurter_api/*.jsonl`, format JSONL, stream `api_frankfurter`
`file_csv_communes`	Source File	URL HTTPS de data.gouv.fr, format csv, option de lecture `{"dtype": "string"}`
`minio_raw_csv`	Destination S3	Bucket `dsid-raw`, chemin `raw/csv/communes`, même endpoint, format JSONL
`s3_minio_csv_communes`	Source S3	Glob `raw/csv/communes/communes_france/*.jsonl`, format JSONL, stream `csv_communes`
`mariadb_espocrm`	Source MySQL	Hôte `172.19.0.1`, port 3306, base `espocrm`, utilisateur `airbyte` en lecture seule, méthode « Scan Changes with User Defined Cursor »
`postgres_bronze`	Destination Postgres	Hôte `172.19.0.1`, port 5432, base `warehouse`, schéma par défaut `bronze`

Deux réglages ont une vraie conséquence sur les données. L'option `dtype=string` de la source File empêche Airbyte de deviner les types : sans elle, le code INSEE `01001` serait lu comme le nombre 1001. Avec elle, tout arrive en texte et c'est Silver qui type. Ensuite, c'est le nom du stream de la source S3 qui donne le nom de la table

Bronze : `api_frankfurter` et `csv_communes`, ce qui respecte les formes `bronze.api_<source>` et `bronze.csv_<source>` demandées.

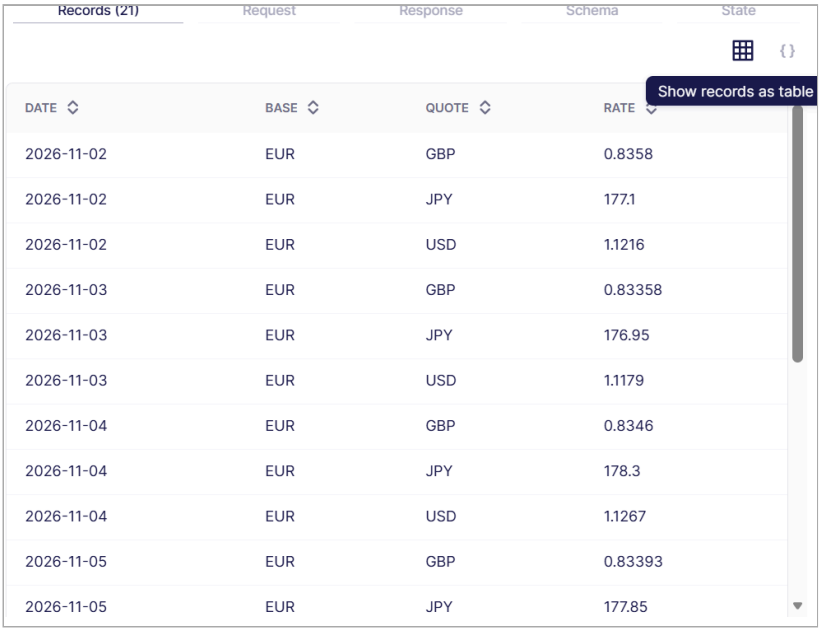


Figure 2 – Test du connecteur dans le Builder d'Airbyte. L'appel GET renvoie 21 enregistrements avec les champs date, base, quote et rate : l'API répond et la structure est bien celle du contrat.

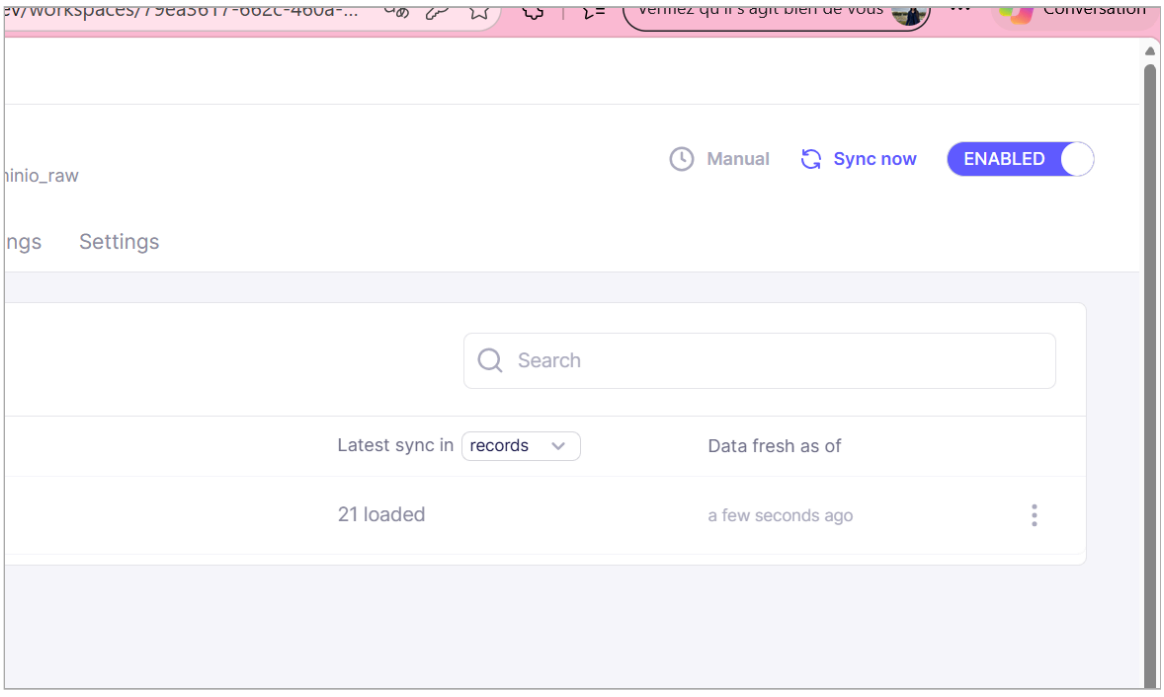


Figure 3 – Connexion api_frankfurter_to_minio : statut « Synced », 21 enregistrements chargés. Le flux API → Airbyte → MinIO fonctionne.

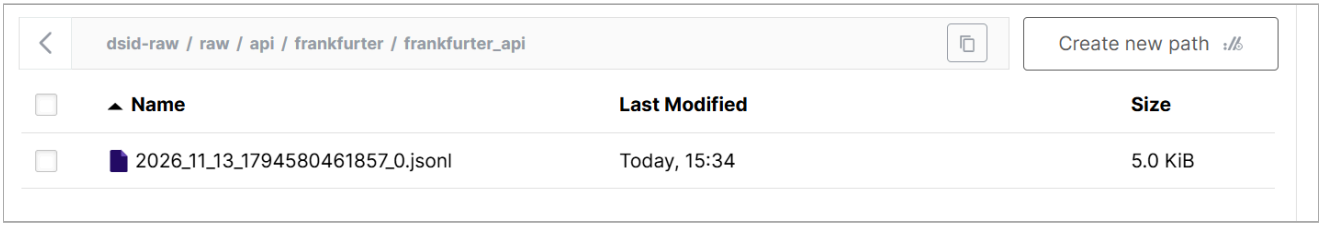


Figure 4 – Console MinIO : le fichier 2026_11_13_1794580461857_0.jsonl (5,0 Kio) est présent dans dsid-raw/raw/api/frankfurter/frankfurter_api. C'est le lot brut de l'API, daté dans son nom.

3.2 Convention de lot dans MinIO

Tous les fichiers bruts sont dans le bucket privé `dsid-raw`. C'est Airbyte qui nomme les fichiers, sous la forme `<chemin de la destination>/<stream>/<AAAA_MM_JJ>_<horodatage en ms>_<numéro>.jsonl`. Le nom contient donc la date du lot. Comme les deux connexions vers MinIO sont en mode Append, une nouvelle synchronisation ajoute un nouveau fichier daté et ne touche pas aux anciens.

Pour le CSV, j'ai déposé en plus le fichier original dans `raw/originaux/csv_communes/2026-11-13_communes_france.csv`, sans le modifier ni l'ouvrir dans un tableur. Je l'ai placé en dehors des dossiers lus par les sources S3 pour qu'il ne soit pas rechargé par erreur.

Le manifeste est produit par le script `scripts/minio_manifest.py`. Il relit chaque objet dans MinIO, calcule son SHA-256 et écrit `data/manifestes/minio_raw_manifest.csv` (et la même chose en JSON).

Tableau 8 – Manifeste de la zone RAW (bucket dsid-raw), généré le 13 novembre 2026

Clé de l'objet	Taille (octets)	Date (UTC)	SHA-256
raw/api/frankfurter/frankfurter_api/2026_11_13_1794580461857_0.jsonl	5 168	13/11 14:34:22	b656cf2361561e5ef473b1ef0b7e2d46af09b5cab2a9c8c8d3711c7008300c20
raw/csv/communes/communes_france/2026_11_13_1794593861115_0.jsonl	42 388 159	13/11 18:18:04	3a30479f913739a544d53753f9ef403c3f5a67e449e96f38a943765ccc6d3e6d
raw/csv/communes/communes_france/2026_11_13_1794593861115_1.jsonl	31 496 312	13/11 18:18:05	52f4c44b57213192fe5f7a8c6c345dff8ca2d25bbb7b48012efac22903b95bf7
raw/originaux/csv_communes/2026-11-13_communes_france.csv	21 344 470	13/11 19:42:13	f15a1dffa3105d267a4aaa4e3e7b161fea485187a6726bdf59247a94c70552e6

Le SHA-256 du CSV original relu dans MinIO est identique à celui calculé juste après le téléchargement (tableau 4) : le fichier déposé est bien l'original, au bit près. Airbyte a découpé le lot CSV en deux fichiers JSONL (20 000 et 14 868 enregistrements). Ils sont plus gros que le CSV parce que chaque ligne JSONL répète le nom des 62 colonnes.

3.3 Structure observée et provenance

Pour les deux flux qui passent par MinIO, la table Bronze a 7 colonnes : les 4 colonnes techniques d'Airbyte (`_airbyte_raw_id`, `_airbyte_extracted_at`, `_airbyte_meta`, `_airbyte_generation_id`), la colonne `_airbyte_data` de type jsonb, et deux colonnes sur le fichier d'origine (`_ab_source_file_url` et `_ab_source_file_last_modified`).

Je ne m'attendais pas à cette forme. Les fichiers JSONL écrits par la destination S3 emballent chaque enregistrement dans un champ `_airbyte_data`, et la source S3 le relit tel quel. Le contenu métier (date, base, quote et rate pour l'API, les 62 colonnes pour le CSV) se trouve donc dans la colonne jsonb `_airbyte_data`. Pour une couche Bronze ce n'est pas gênant, c'est même cohérent : rien n'est interprété ni corrigé à ce stade, et c'est Silver qui extrait et type les champs.

Pour le CRM, la source MySQL crée une colonne par colonne MariaDB, plus les 4 colonnes techniques : 31 colonnes pour `account`, 25 pour `contact`, 25 pour `opportunity`, 10 et 9 pour les deux tables de relation.

Chaque ligne Bronze permet de remonter à sa source : le fichier MinIO pour les flux A et B, le numéro du job Airbyte (`_airbyte_meta ->> 'sync_id'`) et la date d'extraction (`_airbyte_extracted_at`). Voici ce que donne la requête de provenance du fichier `sql/tests/bronze_controles.sql`.

Tableau 9 – Provenance des lignes Bronze

Table Bronze	Fichier d'origine (_ab_source_file_url)	Job Airbyte	Extrait le (UTC)	Lignes

api_frankfurter	raw/api/frankfurter/frankfurter_api/2026_11_13_1794580461857_0.jsonl	8	13/11/2026 17:42:30	21
csv_communes	raw/csv/communes/communes_france/2026_11_13_1794593861115_0.jsonl	11	13/11/2026 18:37:07	20 000
csv_communes	raw/csv/communes/communes_france/2026_11_13_1794593861115_1.jsonl	11	13/11/2026 18:37:13	14 868
Les 5 tables CRM	Lecture directe de MariaDB, sans fichier	16	13/11/2026 19:21:47	1, 2, 1, 2, 2

3.4 Contrôles

J'ai compté pour chaque table Bronze les lignes, les clés distinctes et les clés nulles. Exemple de requête pour le CSV (les autres sont sur le même modèle dans `sql/tests/bronze_controles.sql`, résultat complet dans `preuves/controles_bronze.txt`) :

```
SELECT COUNT(*)                                          AS lignes,
       COUNT(DISTINCT _airbyte_data ->> 'code_insee')    AS cles_distinctes,
       COUNT(*) FILTER (WHERE NULLIF(TRIM(_airbyte_data ->> 'code_insee'), '') IS NULL) AS cles_nulles
FROM bronze.csv_communes;
```

Tableau 10 – Contrôles Bronze

Table Bronze	Clé contrôlée	Lignes	Clés distinctes	Clés nulles	Doublons
api_frankfurter	(date, base, quote)	21	21	0	0
csv_communes	code_insee	34 868	34 868	0	0
account	id	1	1	0	0
contact	id	2	2	0	0
opportunity	id	1	1	0	0
account_contact	id	2	2	0	0
contact_opportunity	id	2	2	0	0

Comparaison entre la source et Bronze. Pour l'API, j'attendais 7 jours ouvrés × 3 devises = 21 lignes et j'en ai 21. Pour le CSV, le fichier fait 34 869 lignes dont une d'en-tête, soit 34 868 communes, et Bronze contient 34 868 lignes. Pour le CRM, j'ai saisi 1 compte, 2 contacts et 1 opportunité, et je retrouve 1, 2 et 1 lignes, plus 2 liens dans chaque table de relation. Il n'y a aucun écart à expliquer.

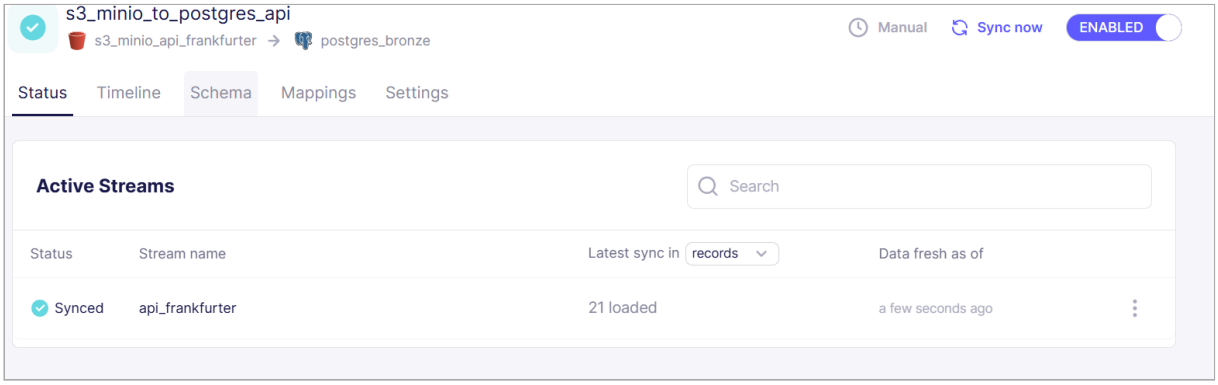


Figure 5 – Connexion s3_minio_to_postgres_api : 21 lignes chargées dans bronze.api_frankfurter. Le flux MinIO → Airbyte → PostgreSQL Bronze fonctionne.

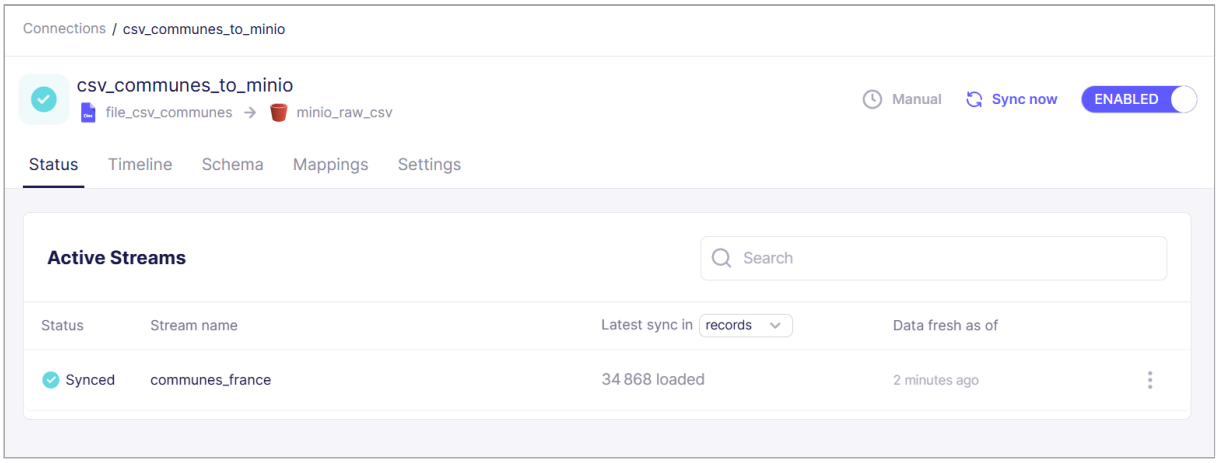


Figure 6 – Connexion csv_communes_to_minio : 34 868 enregistrements déposés dans MinIO, ce qui correspond au nombre de communes du fichier.

```
@melinaadel →/workspaces/ADEL_MELINA_PROJET_BIG_DATA (main) $ docker
 exec -i dsid-postgres psql -P pager=off -U warehouse -d warehouse -c
 "select count(*) as nb_lignes from bronze.csv_communes;"
 nb_lignes
-----------
     34868
(1 row)

@melinaadel →/workspaces/ADEL_MELINA_PROJET_BIG_DATA (main) $
```

Figure 7 – Comptage dans PostgreSQL après la synchronisation MinIO → Bronze : bronze.csv_communes contient 34 868 lignes.

3.5 Que se passe-t-il si on rejoue un lot ?

- **Source → MinIO (Full refresh | Append).** Relancer la synchronisation ajoute un nouveau fichier daté dans MinIO. Le lot précédent reste en place.
- **MinIO → Bronze (Incremental | Append sur la date du fichier).** Seuls les fichiers nouveaux sont chargés. Je l'ai vérifié en relançant `s3_minio_to_postgres_csv` sans nouveau fichier : 0 ligne chargée (figure 8) et la table est restée à 34 868 lignes. Si un nouveau lot arrive, ses lignes s'ajoutent en Bronze. La même clé peut alors exister plusieurs fois : c'est normal en Bronze, le test 3 compte ces doublons et Silver garde la version la plus récente.
- **CRM.** Les entités sont en incrémental sur `modified_at` et les relations sont relues en entier à chaque fois (partie 6).

Je ne fais aucun UPDATE manuel dans Bronze. Toutes les corrections sont dans Silver.

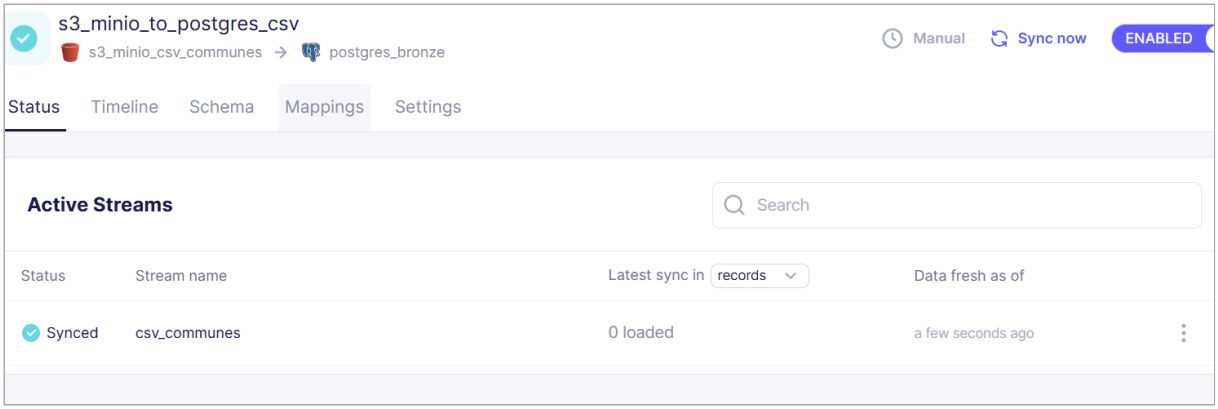


Figure 8 – Relance de s3_minio_to_postgres_csv sans nouveau fichier dans MinIO : « 0 loaded ». Le mode incrémental ne recharge pas un lot déjà chargé.

4. Silver

La couche Silver est écrite en SQL PostgreSQL, dans trois fichiers versionnés du dossier `sql/silver/` : `api_frankfurter_rates.sql`, `csv_communes.sql` et `crm_espocrm.sql`. J'ai choisi de faire des vues (`CREATE OR REPLACE VIEW`) plutôt que des tables. Une vue est toujours à jour par rapport à Bronze, on peut rejouer le script autant de fois qu'on veut, et toute la transformation se lit dans le fichier. Avec 34 868 lignes au maximum, le temps de calcul n'est pas un sujet.

4.1 Règles de nommage et de types

Tableau 11 – Convention appliquée

Concept	Règle	Exemples dans mon projet
Nom de table	`silver.<source>_<entité>`	`silver.crm_account`, `silver.csv_communes`, `silver.api_frankfurter_rates`
Nom de colonne	snake_case	`exchange_rate`, `modified_at_utc`
Identifiant métier	`text`	`code_insee` (« 01001 » reste « 01001 »), `account_id`
Date sans heure	`date`	`rate_date`, `close_date`
Horodatage	`timestamptz`	`ingested_at_utc`, `created_at_utc`
Entier	`integer`	`population`, `probability`
Montant ou décimal	`numeric(p,s)`	`amount numeric(18,2)`, `exchange_rate numeric(18,6)`, `latitude_mairie numeric(9,6)`
Booléen	`boolean`	`is_deleted`, `is_inactive`
Texte libre	`text`	`account_name`, `nom_standard`

4.2 Les 7 vues

Tableau 12 – Vues Silver, grain et clé

Vue Silver	Source Bronze	Grain	Clé	Colonnes
`silver.api_frankfurter_rates`	`bronze.api_frankfurter`	Un taux par date et devise	(rate_date, base_currency, quote_currency)	7
`silver.csv_communes`	`bronze.csv_communes`	Une commune	code_insee	65
`silver.crm_account`	`bronze.account`	Un compte	account_id	13
`silver.crm_contact`	`bronze.contact`	Un contact	contact_id	10
`silver.crm_opportunity`	`bronze.opportunity`	Une opportunité	opportunity_id	15
`silver.crm_account_contact`	`bronze.account_contact`	Un lien compte-contact	(account_id, contact_id)	8
`silver.crm_contact_opportunity`	`bronze.contact_opportunity`	Un lien contact-opportunité	(contact_id, opportunity_id)	7

4.3 Règles de transformation

- **Texte.** `TRIM`, puis une chaîne vide devient `NULL`. Je justifie ce choix : dans ces trois sources, une chaîne vide ne veut rien dire de plus qu'une valeur absente (un site web non renseigné dans EspoCRM, un code d'aire d'attraction vide dans le CSV). Les codes devise sont mis en majuscules.

- **Nombres et dates venant du jsonb (API et CSV).** La valeur est convertie seulement si elle a le bon format, vérifié par une expression régulière. Sinon elle devient `NULL`. Pour ne pas perdre de valeurs en silence, un test supplémentaire (8b) compte les valeurs non vides qui n'ont pas pu être converties.
- **Identifiants.** Gardés en texte : `code_insee`, `dep_code`, `reg_code`, `code_postal` et les identifiants EspoCRM.
- **Décimaux.** `numeric(18,2)` pour les surfaces, densités, altitudes et montants, `numeric(9,6)` pour les latitudes et longitudes, `numeric(18,6)` pour les taux de change.
- **Horodatages du CRM.** EspoCRM les enregistre en UTC sans fuseau. Silver les déclare en UTC (`AT TIME ZONE 'UTC'`) pour obtenir un `timestamptz`.
- **Suppressions logiques du CRM.** Les lignes `deleted = true` sont gardées, avec une colonne `is_deleted`. Je préfère ne pas filtrer en silence : celui qui utilise la vue décide.
- **Lignes sans clé.** Elles sont exclues de Silver et comptées dans le test 6.

Tableau 13 – Dictionnaire des renommages

Source	Colonne d'origine	Colonne Silver	Justification
API	`date`	`rate_date`	« date » est un mot réservé et trop vague
API	`base`, `quote`, `rate`	`base_currency`, `quote_currency`, `exchange_rate`	Préciser qu'il s'agit de devises et d'un taux de change
CSV	Les 62 colonnes	Noms inchangés	Déjà en snake_case et explicites
CRM	`id`	`account_id`, `contact_id`, `opportunity_id`	Même concept, même nom dans toutes les vues du CRM
CRM	`name`, `type`	`account_name`, `opportunity_name`, `account_type`	« name » et « type » seuls sont ambigus
CRM	`deleted`	`is_deleted`	Booléen explicite
CRM	`created_at`, `modified_at`	`created_at_utc`, `modified_at_utc`	Le fuseau est dans le nom
CRM	`role`	`contact_role`	Rôle du contact dans le lien
CRM	`billing_address_city`, `billing_address_country`	`billing_city`, `billing_country`	Plus court, même sens

Je n'ai fait aucun rapprochement entre l'API, le CSV et le CRM : chaque vue ne lit que sa propre source. Par exemple `amount_currency` (CRM) et `quote_currency` (API) restent deux colonnes distinctes, parce qu'elles n'ont pas le même sens.

4.4 Dédoublonnage

Je n'utilise jamais `DISTINCT`. Chaque vue numérote les lignes d'une même clé avec `ROW_NUMBER()` et garde la première selon une règle fixe :

- API et CSV : la ligne extraite le plus récemment (`_airbyte_extracted_at`, puis `_airbyte_raw_id` en cas d'égalité) ;
- entités du CRM : la version modifiée le plus récemment (`modified_at`, puis `_airbyte_extracted_at`, puis `_airbyte_raw_id`) ;
- relations du CRM : la ligne extraite le plus récemment pour le couple de clés.

Le nombre de lignes écartées est donné par le test 6. Aujourd'hui il vaut 0 partout, parce qu'il n'y a qu'un seul lot par source.

4.5 Traçabilité

Les 7 vues ont les trois mêmes colonnes de traçabilité.

Colonne	Contenu
`source_system`	`'api_frankfurter_ecb'`, `'csv_communes_data_gouv'` ou `'crm_espocrm'`
`source_record_id`	Identifiant d'origine : `date|base|quote` pour l'API, le `code_insee` pour le CSV, l'`id` EspoCRM pour le CRM
`ingested_at_utc`	Date d'extraction Airbyte (`_airbyte_extracted_at`), en `timestamptz`

4.6 Exemple de SQL : silver.crm_opportunity

Les 7 vues sont construites sur le même modèle en trois étapes : nettoyer et typer, numéroter les versions d'une même clé, garder la plus récente et ajouter la traçabilité. Voici la vue des opportunités, telle qu'elle est dans `sql/silver/crm_espocrm.sql`.

```
CREATE OR REPLACE VIEW silver.crm_opportunity AS
WITH cleaned AS (
    SELECT
        NULLIF(TRIM(b.id), '')                      AS opportunity_id,
        NULLIF(TRIM(b.name), '')                    AS opportunity_name,
        NULLIF(TRIM(b.stage), '')                   AS stage,
        CAST(b.amount AS numeric(18,2))             AS amount,
        UPPER(NULLIF(TRIM(b.amount_currency), ''))  AS amount_currency,
        CAST(b.probability AS integer)              AS probability,
        CAST(b.close_date AS date)                  AS close_date,
        NULLIF(TRIM(b.lead_source), '')             AS lead_source,
        NULLIF(TRIM(b.account_id), '')              AS account_id,
        b.deleted                                   AS is_deleted,
        b.created_at  AT TIME ZONE 'UTC'            AS created_at_utc,
        b.modified_at AT TIME ZONE 'UTC'            AS modified_at_utc,
        b._airbyte_raw_id,
        b._airbyte_extracted_at
    FROM bronze.opportunity AS b
),
ranked AS (
    SELECT c.*,
           ROW_NUMBER() OVER (
               PARTITION BY c.opportunity_id
               ORDER BY c.modified_at_utc DESC NULLS LAST, c._airbyte_extracted_at DESC,
c._airbyte_raw_id DESC
           ) AS rn
    FROM cleaned AS c
    WHERE c.opportunity_id IS NOT NULL
)
SELECT r.opportunity_id, r.opportunity_name, r.stage, r.amount, r.amount_currency, r.probability,
       r.close_date, r.lead_source, r.account_id, r.is_deleted, r.created_at_utc, r.modified_at_utc,
       'crm_espocrm'::text                          AS source_system,
       r.opportunity_id                             AS source_record_id,
       CAST(r._airbyte_extracted_at AS timestamptz) AS ingested_at_utc
FROM ranked AS r
WHERE r.rn = 1;
```

Pour l'API et le CSV, la seule différence est que les champs sont d'abord extraits du jsonb, par exemple `_airbyte_data ->> 'rate'`, avec un contrôle de format avant la conversion :

```
CASE WHEN NULLIF(TRIM(b._airbyte_data ->> 'rate'), '') ~ '^-?[0-9]+(\.[0-9]+)?$'
     THEN (TRIM(b._airbyte_data ->> 'rate'))::numeric(18,6) END   AS exchange_rate
```

5. Tests

5.1 Méthode

Chaque vue Silver a ses 8 tests obligatoires, écrits dans `sql/tests/` (un fichier par source). Je les exécute avec psql et je garde la sortie complète, c'est-à-dire la requête et son résultat, dans le dossier `preuves/` :

```
docker exec -i dsid-postgres psql -U warehouse -d warehouse -P pager=off -a \
  < sql/tests/crm_espocrm_tests.sql > preuves/tests_silver_crm_espocrm.txt 2>&1
```

Ci-dessous je donne pour chaque test la requête (sur l'exemple de `silver.crm_opportunity`, les autres vues ont la même requête avec leur propre clé), le résultat obtenu pour les 7 vues et ce que j'en conclus.

5.2 Requêtes, résultats et interprétation

Test 1 – Clé non nulle

```
SELECT COUNT(*) AS cles_nulles
FROM silver.crm_opportunity
WHERE opportunity_id IS NULL;
```

Résultat : 0 pour les 7 vues. **Interprétation :** aucune ligne Silver n'a de clé vide. C'est attendu, puisque les vues excluent les lignes sans clé, mais le test le vérifie.

Test 2 – Unicité

```
SELECT opportunity_id, COUNT(*) AS n
FROM silver.crm_opportunity
GROUP BY opportunity_id
HAVING COUNT(*) > 1;
```

Résultat : 0 ligne pour les 7 vues. **Interprétation :** il y a au maximum une ligne par clé, au grain annoncé dans le tableau 12.

Test 3 – Doublons

```
SELECT
  (SELECT COUNT(*) - COUNT(DISTINCT NULLIF(TRIM(id), '')) FROM bronze.opportunity
    WHERE NULLIF(TRIM(id), '') IS NOT NULL)                                  AS doublons_bronze,
  (SELECT COUNT(*) - COUNT(DISTINCT opportunity_id) FROM silver.crm_opportunity) AS doublons_silver;
```

Résultat : 0 doublon en Bronze et 0 doublon en Silver pour les 7 vues. **Interprétation :** Silver n'a aucun doublon. Bronze n'en a pas non plus pour l'instant, car chaque source n'a été chargée qu'une fois. Si un deuxième lot arrive, le premier chiffre augmentera et le second devra rester à 0.

Test 4 – Nommage

```
SELECT column_name
FROM information_schema.columns
WHERE table_schema = 'silver' AND table_name = 'crm_opportunity'
  AND column_name !~ '^[a-z][a-z0-9_]*$';
```

Résultat : 0 ligne pour les 7 vues. **Interprétation :** toutes les colonnes Silver sont en snake_case.

Test 5 – Types

```
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_schema = 'silver' AND table_name = 'crm_opportunity'
ORDER BY ordinal_position;
```

Tableau 14 – Types observés

Vue	Types observés (résultat du test 5)
`api_frankfurter_rates`	`rate_date` date ; `base_currency`, `quote_currency` text ; `exchange_rate` numeric ; `source_system`, `source_record_id` text ; `ingested_at_utc` timestamp with time zone

csv_communes	65 colonnes : 40 text (dont tous les codes), 8 integer, 16 numeric, 1 timestamp with time zone
crm_account	Identifiant et libellés text ; is_deleted boolean ; created_at_utc, modified_at_utc, ingested_at_utc timestamp with time zone
crm_contact	Identifiants et noms text ; is_deleted boolean ; les 3 dates en timestamp with time zone
crm_opportunity	opportunity_id, opportunity_name, stage, amount_currency, lead_source, account_id text ; amount numeric ; probability integer ; close_date date ; is_deleted boolean ; les 3 dates en timestamp with time zone
crm_account_contact	Identifiants, contact_role text ; is_inactive, is_deleted boolean ; ingested_at_utc timestamp with time zone
crm_contact_opportunity	Identifiants, contact_role text ; is_deleted boolean ; ingested_at_utc timestamp with time zone

Interprétation : les types correspondent à la convention du tableau 11. Les identifiants sont bien en text, y compris ceux qui ne contiennent que des chiffres, comme le code INSEE.

Test 6 – Réconciliation Bronze / Silver

```
SELECT bronze_rows, bronze_cles_distinctes, lignes_sans_cle, silver_rows,
       bronze_rows - silver_rows            AS ecart_brut,
       bronze_cles_distinctes - silver_rows AS ecart_inexplique
FROM (
  SELECT
    (SELECT COUNT(*) FROM bronze.opportunity)                                    AS bronze_rows,
    (SELECT COUNT(DISTINCT NULLIF(TRIM(id), '')) FROM bronze.opportunity
      WHERE NULLIF(TRIM(id), '') IS NOT NULL)                                    AS bronze_cles_distinctes,
    (SELECT COUNT(*) FROM bronze.opportunity WHERE NULLIF(TRIM(id), '') IS NULL) AS lignes_sans_cle,
    (SELECT COUNT(*) FROM silver.crm_opportunity)                                AS silver_rows
) AS t;
```

Tableau 15 – Résultat du test 6

Vue Silver	Lignes Bronze	Lignes sans clé	Lignes Silver	Écart
api_frankfurter_rates	21	0	21	0
csv_communes	34 868	0	34 868	0
crm_account	1	0	1	0
crm_contact	2	0	2	0
crm_opportunity	1	0	1	0
crm_account_contact	2	0	2	0
crm_contact_opportunity	2	0	2	0

Interprétation : aucune ligne n'est perdue entre Bronze et Silver. Les deux seules règles qui peuvent créer un écart sont l'exclusion des lignes sans clé et le dédoublonnage, et aujourd'hui aucune des deux ne s'applique. Pour le CRM, la requête sépare l'écart brut (versions anciennes et lignes sans clé, qui est normal) de l'écart inexpliqué, qui doit toujours rester à 0.

Test 7 – Traçabilité

```
SELECT COUNT(*) FILTER (WHERE source_system IS NULL)    AS source_system_null,
       COUNT(*) FILTER (WHERE source_record_id IS NULL) AS source_record_id_null,
       COUNT(*) FILTER (WHERE ingested_at_utc IS NULL)  AS ingested_at_null
FROM silver.crm_opportunity;
```

Résultat : 0, 0 et 0 pour les 7 vues. **Interprétation :** chaque ligne Silver indique sa source, son identifiant d'origine et sa date d'ingestion.

Test 8 – Règle métier

```
SELECT
  COUNT(*) FILTER (WHERE o.amount < 0)                                        AS montant_negatif,
  COUNT(*) FILTER (WHERE o.probability < 0 OR o.probability > 100)            AS probabilite_hors_bornes,
  COUNT(*) FILTER (WHERE o.stage IS NULL OR o.stage NOT IN
     ('Prospecting','Qualification','Proposal','Negotiation','Closed Won','Closed Lost')) AS
etape_hors_domaine,
  COUNT(*) FILTER (WHERE o.account_id IS NOT NULL AND a.account_id IS NULL)   AS compte_introuvable,
  COUNT(*) FILTER (WHERE o.close_date IS NULL)                                AS date_de_cloture_manquante
FROM silver.crm_opportunity AS o
LEFT JOIN silver.crm_account AS a ON a.account_id = o.account_id;
```

Tableau 16 – Règles métier contrôlées et résultats

Vue	Règles contrôlées	Anomalies
`api_frankfurter_rates`	Taux strictement positif ; pas de date dans le futur ; pas de taux un samedi ou un dimanche ; devise de base égale à EUR ; devise cotée dans USD, GBP, JPY	0
`csv_communes`	Code INSEE au bon format (5 chiffres, ou 2A / 2B suivi de 3 chiffres) ; population positive ou nulle ; latitude entre -90 et 90 ; longitude entre -180 et 180	0
`crm_account`	Nom renseigné ; dates présentes ; modification postérieure ou égale à la création ; pas de création dans le futur	0
`crm_contact`	Nom renseigné ; le compte rattaché existe dans `silver.crm_account`	0
`crm_opportunity`	Montant positif ou nul ; probabilité entre 0 et 100 ; étape dans la liste d'EspoCRM ; compte existant ; date de clôture renseignée	0
`crm_account_contact`	Le compte et le contact du lien existent en Silver	0
`crm_contact_opportunity`	Le contact et l'opportunité du lien existent en Silver	0

Interprétation : aucune anomalie. Les contrôles du CRM sur l'existence du compte, du contact ou de l'opportunité restent à l'intérieur du CRM, ce n'est pas un rapprochement entre sources. **Test 8b** (API et CSV) : 0 date et 0 taux non convertis pour l'API, 0 population non convertie pour le CSV. Les contrôles de format de la partie 4.3 n'ont donc transformé aucune valeur en NULL par erreur.

5.3 Synthèse et écarts

Les 8 tests passent sur les 7 vues, soit 56 contrôles, plus le test 8b. Il n'y a aucun écart à ce jour. Je reste prudente sur ce résultat : avec un seul lot par source et des sources propres, c'est logique, et les tests 3 et 6 ne montrent pas encore le dédoublonnage en action. Ils serviront surtout à détecter un problème lors d'un prochain chargement. Les sorties complètes sont dans `preuves/tests_silver_api_frankfurter.txt`, `preuves/tests_silver_csv_communes.txt` et `preuves/tests_silver_crm_espocrm.txt`.

6. CRM delta

L'objectif de cette partie est de prouver qu'une information saisie dans EspoCRM est bien collectée jusqu'en Bronze.

6.1 État initial

Avant de saisir quoi que ce soit, j'ai compté les lignes des trois tables dans MariaDB : 0 compte, 0 contact et 0 opportunité. EspoCRM venait d'être installé, il n'y avait donc aucun identifiant existant à relever.

6.2 Saisie dans EspoCRM

J'ai créé dans l'interface d'EspoCRM (port 8081) : un compte « Société Test Big Data », deux contacts « Alice Test » et « Bob Test » rattachés à ce compte, et une opportunité « Opportunité Test BD » (étape Prospecting, montant 10 000, probabilité 10 %, clôture le 14 novembre 2026) liée au compte et aux deux contacts.

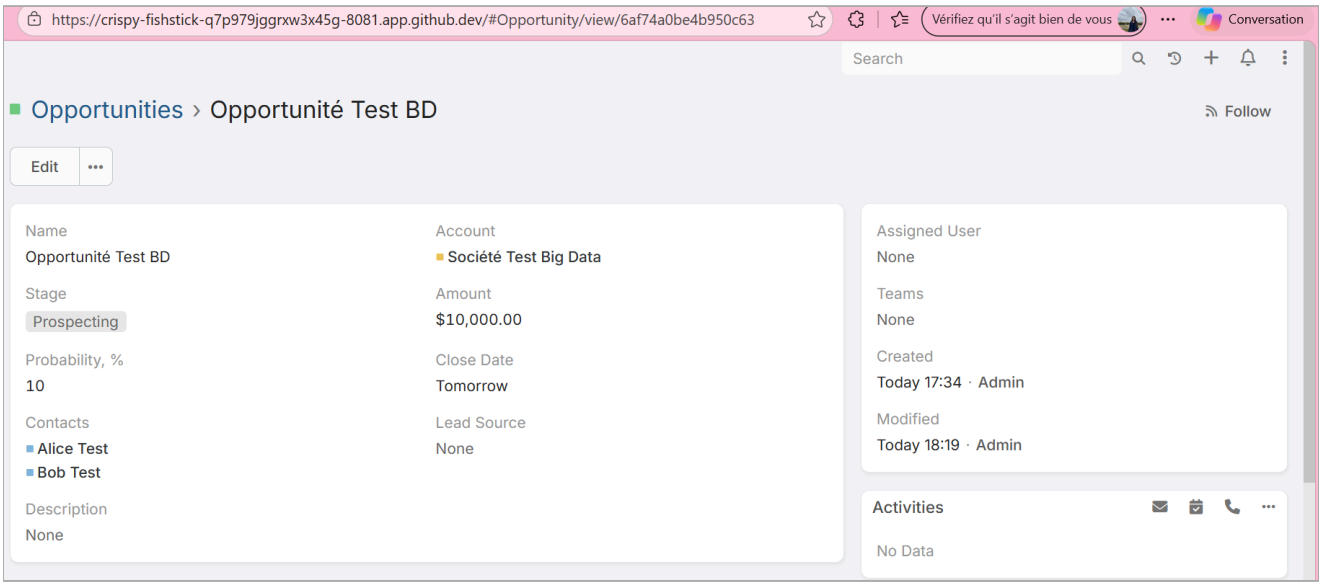


Figure 9 – Fiche de l'opportunité dans EspoCRM. Elle est liée au compte « Société Test Big Data » et aux contacts Alice Test et Bob Test. Son identifiant 6af74a0be4b950c63 est visible dans l'URL, ce qui permet de la retrouver dans MariaDB et dans Bronze.

6.3 État après la saisie

```
@melinaadel ➜/workspaces/ADEL_MELINA_PROJET_BIG_DATA (main) $ docker
 exec -it dsid-mariadb mariadb -uespocrm -p espocrm -e "select 'accou
nt' t, id, name from account union all select 'contact', id, concat(f
irst_name,' ',last_name) from contact union all select 'opportunity',
 id, name from opportunity;"
Enter password:
+-------------+-------------------+-------------------------+
| t           | id                | name                    |
+-------------+-------------------+-------------------------+
| account     | 6af74952ca9701674 | Société Test Big Data   |
| contact     | 6af749a37f6fbc62b | Alice Test              |
| contact     | 6af749c6c38791641 | Bob Test                |
| opportunity | 6af74a0be4b950c63 | Opportunité Test BD     |
+-------------+-------------------+-------------------------+
@melinaadel ➜/workspaces/ADEL_MELINA_PROJET_BIG_DATA (main) $
```

Figure 10 – Requête dans MariaDB après la saisie : les 4 identifiants créés par EspoCRM (1 compte, 2 contacts, 1 opportunité).

Tableau 17 – Avant et après la saisie

Table	MariaDB avant	MariaDB après	Bronze après synchro	Silver
account	0	1	1	1
contact	0	2	2	2
opportunity	0	1	1	1
account_contact	non relevé *	2	2	2
contact_opportunity	non relevé *	2	2	2

* Je n'ai pas compté les tables de relation avant la saisie. Elles étaient forcément vides, puisqu'il n'existait ni compte ni contact à relier. Le « 2 » après la saisie est le nombre de lignes lues dans MariaDB par Airbyte.

6.4 Synchronisation et mode de sync

La connexion `mariadb_espocrm_to_postgres_crm` ne lit que les 5 tables utiles sur les 141 découvertes. Le job n° 16 a réussi le 13 novembre à 19:21 UTC.

Tableau 18 – Mode de synchronisation par table

Table	Mode	Curseur, clé	Justification
`account` et `contact`	Incremental | Append	`modified_at`, `id`	EspoCRM met à jour `modified_at` à chaque modification d'une fiche : c'est un curseur fiable. Seules les fiches nouvelles ou modifiées sont relues et chaque version s'ajoute en Bronze.
`opportunity`	Incremental | Append + Deduped	`modified_at`, `id`	Même curseur. Avec ce mode, Airbyte ne garde en Bronze que la dernière version de chaque `id`.
`account_contact`	Full refresh | Overwrite + Deduped	`id`	Pas de colonne de date, donc pas de curseur possible. La table est relue en entier à chaque synchronisation, ce qui est acceptable pour une petite table de liens.
`contact_opportunity`	Full refresh | Overwrite	aucun	Même raison.

Je dois signaler que les modes ne sont pas parfaitement homogènes : Append pour `account` et `contact` mais Append + Deduped pour `opportunity`, et deux variantes d'Overwrite pour les relations. Je m'en suis rendu compte en relisant la configuration après coup. Cela ne change pas le résultat, parce que les vues Silver dédoublonnent de toute façon par clé, mais dans une prochaine version je mettrais le même mode pour les trois entités. Le CDC n'était pas imposé et je ne l'ai pas utilisé.

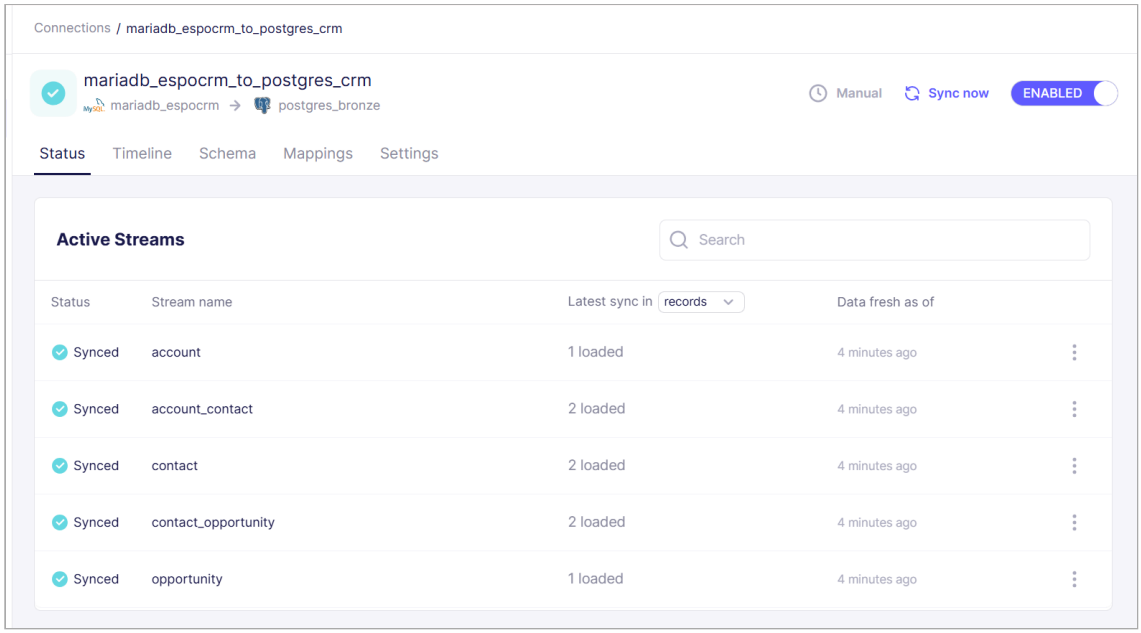


Figure 11 – Connexion mariadb_espocrm_to_postgres_crm après la synchronisation : les 5 flux sont « Synced » avec 1 compte, 2 liens compte-contact, 2 contacts, 2 liens contact-opportunité et 1 opportunité.

6.5 Preuve en Bronze

```
SELECT id, name, modified_at FROM bronze.account;
SELECT id, first_name, last_name, account_id FROM bronze.contact;
SELECT id, name, stage, amount, account_id FROM bronze.opportunity;
SELECT id, account_id, contact_id FROM bronze.account_contact;
SELECT id, contact_id, opportunity_id FROM bronze.contact_opportunity;
```

Tableau 19 – Lignes retrouvées en Bronze

Table Bronze	id	Contenu
`account`	`6af74952ca9701674`	Société Test Big Data, modifié le 2026-11-13 17:30:58
`contact`	`6af749a37f6fbc62b`	Alice Test, account_id = `6af74952ca9701674`
`contact`	`6af749c6c38791641`	Bob Test, account_id = `6af74952ca9701674`
`opportunity`	`6af74a0be4b950c63`	Opportunité Test BD, Prospecting, 10000, account_id = `6af74952ca9701674`
`account_contact`	1 et 2	Le compte `6af74952ca9701674` relié aux contacts `6af749a37f6fbc62b` et `6af749c6c38791641`
`contact_opportunity`	1 et 2	Les contacts `6af749a37f6fbc62b` et `6af749c6c38791641` reliés à l'opportunité `6af74a0be4b950c63`

Les 4 identifiants de la figure 10 se retrouvent à l'identique en Bronze, avec leurs relations.

6.6 Preuve en Silver

La dernière requête de `sql/tests/crm_espocrm_tests.sql` reconstitue le cas complet à partir des vues Silver, en passant par la table de relation :

```
SELECT a.account_id, a.account_name, o.opportunity_id, o.opportunity_name, o.stage, o.amount,
       c.contact_id, c.first_name, c.last_name
FROM silver.crm_account AS a
JOIN silver.crm_opportunity AS o          ON o.account_id = a.account_id
JOIN silver.crm_contact_opportunity AS co ON co.opportunity_id = o.opportunity_id AND co.is_deleted IS
NOT TRUE
JOIN silver.crm_contact AS c              ON c.contact_id = co.contact_id
ORDER BY c.last_name, c.first_name;

    account_id     |     account_name      |  opportunity_id   |  opportunity_name   |    stage    |
amount  |    contact_id     | first_name | last_name
-------------------+-----------------------+-------------------+---------------------+-------------+-----
-----+-------------------+------------+-----------
 6af74952ca9701674 | Société Test Big Data | 6af74a0be4b950c63 | Opportunité Test BD | Prospecting |
10000.00 | 6af749a37f6fbc62b | Alice      | Test
 6af74952ca9701674 | Société Test Big Data | 6af74a0be4b950c63 | Opportunité Test BD | Prospecting |
10000.00 | 6af749c6c38791641 | Bob        | Test
(2 rows)
```

Interprétation : les données saisies dans EspoCRM sont visibles de bout en bout, de MariaDB jusqu'à Silver, avec les mêmes identifiants. La limite de cette preuve est que je n'ai fait qu'une synchronisation après la saisie. Je n'ai pas eu le temps de modifier une fiche puis de relancer pour montrer l'incrémental sur une mise à jour.

7. Docker / Airbyte

7.1 Versions

Les versions sont figées dans `compose/compose.yaml`. Voici la sortie de `docker compose ps` et les images réellement utilisées le 13 novembre 2026.

```
NAME                 IMAGE                                    STATUS
dsid-adminer         adminer:6.1.0                            Up 3 hours
dsid-espocrm         espocrm/espocrm:10.0.8                   Up 3 hours (healthy)
dsid-espocrm-daemon  espocrm/espocrm:10.0.8                   Up 3 hours
dsid-mariadb         mariadb:11.8.9                           Up 3 hours (healthy)
dsid-minio           dsid/minio:RELEASE.2025-10-15T17-29-55Z  Up 3 hours
dsid-postgres        postgres:18.6                            Up 3 hours (healthy)
```

L'image MinIO est construite localement avec `compose/Dockerfile.minio` (commande `docker compose build minio`). Airbyte est en version 2.5.0, installé avec abctl v0.30.4 dans un cluster kind. Le connecteur Source S3 est en version 4.17.1.

7.2 Ports

Service	PostgreSQL	MariaDB	EspoCRM	Adminer	MinIO (API / console)	Airbyte
Port	5432	3306	8081	8082	9000 / 9001	8000

Ce sont les ports par défaut du kit. Dans le Codespace, j'y accède depuis mon navigateur par la redirection de ports de GitHub, qui est privée et liée à mon compte.

7.3 Réseau

Airbyte ne tourne pas dans le Compose. abctl l'installe dans un petit cluster Kubernetes (kind), qui est lui-même un conteneur Docker. Depuis un pod Airbyte, « localhost » ne désigne donc pas mes services. Sous Linux, ce qui est le cas du Codespace, j'ai utilisé l'adresse de la passerelle du réseau Docker du cluster kind, `172.19.0.1`, avec les ports publiés par le Compose : `172.19.0.1:5432` pour PostgreSQL, `172.19.0.1:3306` pour MariaDB et `http://172.19.0.1:9000` pour MinIO. Cette adresse peut changer d'une machine à l'autre, on la retrouve avec `docker network inspect kind`.

7.4 Installation selon le système

Tableau 20 – Installation sous Linux, Windows et macOS

Étape	Linux / Codespace (testé)	Windows	macOS
Docker	Docker Engine, déjà présent dans le Codespace	Docker Desktop avec WSL 2	Docker Desktop
Fichier .env	`cp .env.example .env`	`Copy-Item .env.example .env`	`cp .env.example .env`
Services	`docker compose build minio` puis `docker compose up -d` (identique sur les trois systèmes)		
abctl	`curl -LsfS https://get.airbyte.com | bash -`	Exécutable téléchargé depuis les releases GitHub d'abctl, à ajouter au PATH	`brew tap airbytehq/tap` puis `brew install abctl`
Airbyte	`abctl local install` puis `abctl local credentials` (identique)		
Adresse des services vue depuis Airbyte	Passerelle Docker (`172.19.0.1` chez moi)	`host.docker.internal`	`host.docker.internal`

Je n'ai pu tester de bout en bout que l'installation Linux, dans le Codespace. J'avais commencé sous Windows sur mon ordinateur, mais avec 8 Go de mémoire, Docker Desktop et Airbyte ne tenaient pas ensemble (partie 8.2). Les colonnes Windows et macOS reprennent les indications du kit et de la documentation d'abctl : je ne les ai pas validées moi-même.

8. Reproductibilité

8.1 Procédure complète

C'est la procédure du `README.md`, dans l'ordre. Le détail champ par champ des écrans Airbyte est dans `airbyte/README_AIRBYTE.md`.

1. **Configuration.** Cloner le dépôt, puis `cd compose` et `cp .env.example .env`. Choisir ses propres mots de passe dans `.env`.
2. **Services.** `docker compose build minio`, `docker compose up -d`, puis `docker compose ps` jusqu'à voir les services « healthy ». Au premier démarrage, les scripts de `compose/init/` créent les schémas `bronze` et `silver` dans PostgreSQL et le compte MariaDB `airbyte` en lecture seule.
3. **MinIO.** Dans la console (port 9001), créer le bucket privé `dsid-raw`.
4. **Airbyte.** `abctl local install`, `abctl local credentials`, puis ouvrir le port 8000.
5. **Sources, destinations et connexions.** Créer les éléments des tableaux 6 et 7. Pour l'API, créer le connecteur dans le Builder et le publier. Pour les deux connexions MinIO → PostgreSQL, décocher les 4 champs `_airbyte_*` dans l'onglet Schema (incident 4).
6. **Synchronisations, dans cet ordre.** `api_frankfurter_to_minio`, `s3_minio_to_postgres_api`, `csv_communes_to_minio`, `s3_minio_to_postgres_csv`. Saisir ensuite les données dans EspoCRM (port 8081) et lancer `mariadb_espocrm_to_postgres_crm`.
7. **Silver.** Exécuter les trois scripts de `sql/silver/` :

```
for f in api_frankfurter_rates csv_communes crm_espocrm; do
  docker exec -i dsid-postgres psql -U warehouse -d warehouse -v ON_ERROR_STOP=1 < sql/silver/$f.sql
done
```

8. **Contrôles et tests.** Exécuter les scripts de `sql/tests/` de la même façon et comparer avec les sorties du dossier `preuves/`.
9. **Manifeste.** `pip install boto3`, puis `python3 scripts/minio_manifest.py --upload-original data/communes_france.csv`.

8.2 Incidents rencontrés

C'est la partie qui m'a pris le plus de temps, bien plus que l'écriture du SQL. Je liste les incidents dans l'ordre où je les ai rencontrés, avec la cause que j'ai trouvée et ce que j'ai fait.

Tableau 21 – Incidents, causes et solutions

N°	Problème	Cause	Solution
1	Impossible de faire tourner toute la stack sur mon ordinateur	8 Go de mémoire seulement : Docker Desktop et le cluster d'Airbyte ne tenaient pas ensemble.	J'ai déplacé le projet dans un GitHub Codespace (4 cœurs, 16 Go). Le dépôt garde la trace de ce passage (commit « Etat avant Codespaces »).
2	Publication du connecteur du Builder : `errors.http.internalServerError`	Dans les logs du serveur Airbyte, l'erreur parlait de « major version 7 ». Le manifeste du Builder est en version majeure 7, mais la table `declarative_manifest_image_version` de la base interne d'Airbyte ne contenait que les versions 0 à 4. C'est un problème déjà signalé sur le GitHub d'Airbyte (issue n° 84928).	J'ai ajouté la ligne manquante (version 7.47.0) avec un `INSERT` dans la base d'Airbyte (figure 12). La publication a fonctionné ensuite. La commande est dans a `irbyte/README_AIRBYTE.md`.

3	Source et destination S3 refusées : « The request signature we calculated does not match »	J'avais mis comme endpoint l'URL publique HTTPS du port 9000 du Codespace. La requête signée passe alors par le proxy de GitHub et la signature n'est plus reconnue par MinIO.	Endpoint interne `http://172.19.0.1:9000`. La source S3 v4.17.1 l'a accepté en HTTP, alors que le kit prévenait que ce n'était pas garanti. J'ai remis le port 9000 en privé.
4	Synchronisation MinIO → PostgreSQL en échec : `column "_airbyte_raw_id" specified more than once`	Les fichiers JSONL écrits par la destination S3 contiennent déjà les champs techniques `_airbyte_raw_id`, `_airbyte_extracted_at`, `_airbyte_generation_id` et `_airbyte_meta`. La destination Postgres ajoute les siens avec les mêmes noms, d'où la collision.	Dans l'onglet Schema de la connexion, j'ai décoché ces 4 champs et gardé les 3 autres (`_airbyte_data`, `_ab_source_file_url`, `_ab_source_file_last_modified`). J'ai refait la même erreur pour le CSV avant d'y penser.
5	Synchronisation du CRM qui ne démarre pas (pod `replication-job` en « Pending »)	`kubectl describe pod` indiquait « 0/1 nodes are available: 1 Insufficient cpu ». Le pod réservait 4 CPU (2 pour l'orchestrateur, 1 pour la source, 1 pour la destination) sur une machine de 4 cœurs où Airbyte en occupe déjà 1,1.	J'ai réduit la réservation de cette connexion (0,5 CPU demandé, 1 CPU maximum) avec un `UPDATE` de la colonne `resource_requirements` dans la base d'Airbyte. Le job suivant a démarré tout de suite et a fini en 72 secondes.
6	Tables Bronze du CRM sans préfixe	Je voulais le préfixe `crm_`, mais il n'a pas été enregistré à la création de la connexion.	J'ai gardé en Bronze les noms d'origine de MariaDB, ce qui reste conforme (le schéma `bronze` est demandé pour ce flux, pas un nom précis), et le préfixe `crm_` est appliqué en Silver.

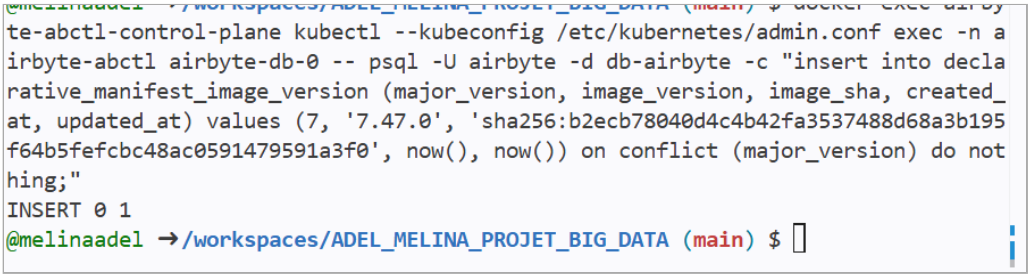


Figure 12 – Correction de l'incident 2 : l'INSERT de la version 7 dans la base interne d'Airbyte renvoie « INSERT 0 1 ». Le connecteur a pu être publié juste après.

J'ai aussi fait deux erreurs de manipulation plus simples : le champ Globs de la source S3 resté vide (il faut valider la valeur avec la touche Entrée) et une ligne de commande collée par erreur dans un fichier de tests SQL, que le message d'erreur de psql m'a permis de repérer.

8.3 Limites

- **Un seul lot par source.** Le dédoublonnage de Silver et l'incrémental du CRM sur une fiche modifiée ne sont pas démontrés avec des données réelles. J'ai seulement vérifié qu'une relance sans nouveau fichier ne recharge rien.
- **Silver en vues.** Elles sont recalculées à chaque lecture. Pour de gros volumes il faudrait des tables.
- **Configuration Airbyte faite à la main.** Elle est décrite dans le README Airbyte mais pas exportée sous forme de fichier. Les deux corrections dans la base interne d'Airbyte (incidents 2 et 5) sont à refaire sur une nouvelle installation.
- **Suppressions dans le CRM.** EspoCRM fait des suppressions logiques, que Silver expose avec `is_deleted`. Une suppression physique faite directement dans MariaDB ne serait pas vue par l'incrémental à curseur.
- **Adresse 172.19.0.1.** Elle dépend de la machine et doit être vérifiée sur un autre poste.
- **Systèmes.** Seule l'installation Linux / Codespace a été testée de bout en bout.
- **Licence de l'API.** La documentation de Frankfurter ne précise pas de licence pour les données.

8.4 Reprise

- **Après un arrêt du Codespace.** `cd compose && docker compose up -d`, puis attendre que les pods Airbyte soient « Running » (`kubectl --kubeconfig ~/.airbyte/abctl/abctl.kubeconfig -n airbyte-abctl get pods`). Les données sont conservées dans les volumes Docker.
- **Après une modification du SQL.** Rejouer le script concerné (`CREATE OR REPLACE VIEW`), puis les tests.
- **Rejouer une synchronisation.** Le comportement est décrit dans la partie 3.5 : ajout d'un fichier daté dans MinIO, chargement des seuls fichiers nouveaux en Bronze, dédoublonnage en Silver.
- **Repartir de zéro.** `docker compose down -v` supprime les volumes, donc aussi les données saisies dans EspoCRM. Il faut ensuite reprendre la procédure 8.1 depuis l'étape 2.

Pour finir, ce que je retiens de ce projet : la partie SQL était la plus rapide, et l'essentiel du temps est passé dans la mise en route des outils et la lecture des messages d'erreur d'Airbyte. Si je devais recommencer, je commencerais directement dans le Codespace et je noterais chaque réglage au moment où je le fais, au lieu de reconstituer la configuration à la fin.